Rupeeflow — AWS Cost Optimization & Proper Setup Guide

Current Spend: ~$900/month **Target Spend:** ~$280–350/month (save $550–620/month) **Current Stack:** m6a.large EC2 · RDS Multi-AZ · Redis on EC2 (Docker) **Date:** March 2026

Part 1 — Where Your $900 Is Going Right Now

Before fixing anything, understand your bill. Go to: **AWS Console → Billing → Cost Explorer → Service breakdown (last 30 days)**

Here is what you will almost certainly find:

```
SERVICE                     ESTIMATED COST      WHY IT'S HIGH
──────────────────────────────────────────────────────────────
RDS (Multi-AZ)              $300 – $500/month   You're paying for
                                                 2 DB servers when
                                                 you only need 1

NAT Gateway                 $80 – $150/month    AWS charges $0.045
                                                 per GB + $0.045/hr
                                                 just to exist

EC2 m6a.large               $73/month           On-demand pricing
                                                 (40% savings with
                                                 reserved)

Data Transfer Out           $50 – $100/month    Every API response
                                                 leaving AWS costs
                                                 $0.09/GB

RDS Storage (gp2 IOPS)      $30 – $80/month     gp2 is old + expensive
                                                 gp3 is 20% cheaper

SES + SNS + S3 + CW         $40 – $70/month     Usually fine, minor
                                                 optimization possible

CloudWatch Logs             $20 – $50/month     Logs never expire =
                                                 infinite accumulation
──────────────────────────────────────────────────────────────
TOTAL                       ~$593 – $1,023/month
```

Part 2 — Immediate Savings (Do These This Week)

Fix 1 — RDS Multi-AZ → Single-AZ + Daily Snapshot

Saves: $150–250/month immediately

What Multi-AZ does: AWS runs TWO identical database servers simultaneously. If one fails, it auto-switches to the second one (failover in ~60 seconds).

Do you need it right now? You are running on a SINGLE EC2 instance. If your EC2 goes down, your app is already down — Multi-AZ RDS doesn't help you. You're paying double for a failover that can't save you when your app server itself is the failure point.

Safer alternative: Daily automated snapshots + Point-in-time recovery. RDS already does this automatically. If your DB has a problem, you restore from the last snapshot (5–15 minutes of downtime, acceptable for your current scale).

How to switch:

```
AWS Console → RDS → Your DB instance → Modify
→ Availability & durability
→ Change "Multi-AZ DB instance" to "Single-AZ DB instance"
→ Apply during next maintenance window (or immediately for staging)
```

Important: Switching from Multi-AZ to Single-AZ causes 1–2 minutes of downtime. Do this during off-peak hours (e.g., 2–4 AM IST Sunday).

Fix 2 — RDS Storage: gp2 → gp3

Saves: $20–40/month

gp2 pricing: $0.115/GB/month gp3 pricing: $0.092/GB/month + baseline 3,000 IOPS free (vs gp2 where IOPS scale with size)

For a 200GB database:

- gp2: $23/month (and if you provisioned extra IOPS, much more)
- gp3: $18.4/month with 3x better baseline performance

```
AWS Console → RDS → Your DB instance → Modify
→ Storage
→ Storage type: gp3
→ Storage: keep same size
→ IOPS: 3000 (default, free)
→ Throughput: 125 MB/s (default, free)
→ Apply immediately ✓ (no downtime for storage type change)
```

Fix 3 — CloudWatch Log Retention: Never → 30 Days

Saves: $15–40/month

By default, CloudWatch log groups **never expire**. You are accumulating logs from every PM2 process, every NestJS request, every RDS slow query. At $0.50/GB/month, 6 months of logs from 6 PM2 processes adds up fast.

Set retention on ALL log groups to 30 days:

```
# Run this in AWS CloudShell or your terminal (AWS CLI)
# Lists all log groups with their retention settings
aws logs describe-log-groups \
  --query "logGroups[*].[logGroupName,retentionInDays]" \
  --output table
```

```
# Set 30-day retention on each log group (run for each group name)
aws logs put-retention-policy \
  --log-group-name "/aws/rds/instance/your-db-name/postgresql" \
  --retention-in-days 30

aws logs put-retention-policy \
  --log-group-name "/rupeeflow/app" \
  --retention-in-days 30

# One-liner: set 30 days on ALL log groups
aws logs describe-log-groups --query "logGroups[*].logGroupName" --output text | \
  tr '\t' '\n' | \
  xargs -I {} aws logs put-retention-policy \
    --log-group-name {} \
    --retention-in-days 30
```

Fix 4 — NAT Gateway: Check If You're Using It

Saves: $80–150/month if you remove it

NAT Gateway is the most common hidden AWS cost. It costs:

- $0.045/hour just to exist = **$32.4/month baseline**
- $0.045/GB for every byte of data passing through it

Check if you have one:

```
AWS Console → VPC → NAT Gateways
```

What NAT Gateway is for: It lets resources in a **private subnet** (no public IP) access the internet. Your EC2 needs internet access to call external PGs (ONIK, GeoPay, etc.) and AWS services (SES, SNS, S3).

Do you need it?

Your resource	In private subnet?	Need NAT?
EC2 (app)	Usually public subnet	NO
RDS	Private subnet	NO — RDS only needs VPC-internal access
Redis on EC2	Same EC2	NO

If your EC2 is in a public subnet (has a public IP), you don't need NAT Gateway at all. Check:

```
AWS Console → EC2 → Your instance → Networking tab
→ If it shows "Public IPv4 address" = it's in a public subnet = no NAT needed
```

If RDS is the only thing in a private subnet (common default setup), you can:

1. Keep RDS in private subnet (correct — DB should never be public)
2. Access RDS from your EC2 via the **private VPC IP** (no NAT needed for VPC-internal traffic)
3. Delete the NAT Gateway

Check your RDS connection string in `.env`:

```
DB_HOST=rupeeflow-db.xxxx.ap-south-1.rds.amazonaws.com
```

This hostname resolves to a **private IP inside your VPC** when accessed from your EC2. No NAT Gateway needed. Your EC2 connects directly.

To confirm RDS doesn't need NAT:

```
# SSH into your EC2 and test direct connection
psql -h rupeeflow-db.xxxx.ap-south-1.rds.amazonaws.com -U your_user -d your_db
# If this works, you're using private VPC routing — NAT not needed for DB
```

Deleting NAT Gateway:

```
AWS Console → VPC → NAT Gateways → Select → Actions → Delete
```

Note: After deleting, also release the associated Elastic IP (EIP) to stop the $3.65/month EIP charge.

Fix 5 — EC2 Reserved Instance (1-Year)

Saves: $28/month (~40% off)

You're on m6a.large at ~$73/month (on-demand). With a 1-year no-upfront reservation:

- Reserved price: ~$44/month
- Saving: $29/month = **$348/year**

Only do this if you're confident you'll keep this instance for 12+ months.

```
AWS Console → EC2 → Reserved Instances → Purchase Reserved Instances
→ Platform: Linux/UNIX
→ Instance type: m6a.large
→ Region: ap-south-1 (Mumbai)
→ Term: 1 Year
→ Payment: No Upfront (pay monthly)
→ Offering class: Standard
```

Part 3 — Your Correct AWS Architecture (Current Scale)

This is what your single-EC2 setup should look like, properly configured:

```
                    ┌──────────────────────────────────────┐
                    │          ap-south-1 (Mumbai)         │
                    │                                      │
                    │  ┌──────────────────────────────┐   │
                    │  │        PUBLIC SUBNET         │   │
                    │  │                              │   │
  Internet ──────────►  EC2 m6a.large                 │   │
  (HTTPS:443)       │  │  ├── NestJS (PM2 × 6)        │   │
                    │  │  ├── Redis (Docker)          │   │
                    │  │  └── PgBouncer (Docker) :6432 │   │
```

```
│  │                                    │   │
│  │  Security Group:                   │   │
│  │  ✓ 443 from 0.0.0.0/0 (HTTPS)    │   │
│  │  ✓ 22 from YOUR_IP only (SSH)    │   │
│  │  ✗ 3000 NOT open (NestJS port)   │   │
│  │  ✗ 5432 NOT open (no direct DB)  │   │
│  └──────────────────┬─────────────────┘   │
│                     │ VPC private routing  │
│  ┌──────────────────▼─────────────────┐   │
│  │        PRIVATE SUBNET              │   │
│  │                                    │   │
│  │  RDS PostgreSQL (Single-AZ)        │   │
│  │  db.t3.large or db.m6g.large       │   │
│  │                                    │   │
│  │  Security Group:                   │   │
│  │  ✓ 5432 from EC2 security group    │   │
│  │  ✗ 5432 NOT open to internet       │   │
│  └────────────────────────────────────┘   │
└──────────────────────────────────────────┘
```

Key points:

- EC2 has a public IP — internet traffic comes directly to it
- RDS is in a private subnet — only reachable from inside your VPC
- No NAT Gateway needed
- No Load Balancer needed (add later when you add a second EC2)
- Nginx on the EC2 handles SSL termination, proxies to NestJS on port 3000

Part 4 — Nginx on EC2 (Replace Your Current Setup)

If you're currently exposing NestJS directly on port 3000 or using a different proxy, here's the correct setup:

```
# /etc/nginx/sites-available/rupeeflow
server {
    listen 80;
    server_name api.rupeeflow.in;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.rupeeflow.in;

    # SSL via Let's Encrypt (free)
    ssl_certificate     /etc/letsencrypt/live/api.rupeeflow.in/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.rupeeflow.in/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # Rate limiting at Nginx level (before NestJS throttler)
    limit_req_zone $binary_remote_addr zone=api:10m rate=50r/s;
```

```
    limit_req zone=api burst=100 nodelay;

    # Real IP from proxy headers (fixes webhook IP guard — S-3 from audit)
    real_ip_header    X-Forwarded-For;
    real_ip_recursive on;
    # Add your known proxy IPs here if behind AWS ALB later:
    # set_real_ip_from 10.0.0.0/8;

    location / {
        proxy_pass         http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection 'upgrade';
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 30s;
        proxy_connect_timeout 10s;
        proxy_send_timeout 30s;

        # Max body size (match NestJS 10mb limit)
        client_max_body_size 10m;
    }

    # Block common attack paths
    location ~* \.(git|env|htaccess|htpasswd)$ {
        deny all;
        return 404;
    }
}
```

Install Nginx and SSL:

```
sudo apt update
sudo apt install -y nginx certbot python3-certbot-nginx

# Get free SSL cert
sudo certbot --nginx -d api.rupeeflow.in

# Nginx auto-renews SSL — verify with:
sudo certbot renew --dry-run

# Enable and start
sudo nginx -t
sudo systemctl enable nginx
sudo systemctl restart nginx
```

Part 5 — EC2 Security Group (What Should Be Open)

This is critical. Many people leave ports wide open. Here is exactly what your EC2 security group should look like:

```
INBOUND RULES — EC2 App Server

┌──────────────────────────────────────────────────────────┐
│ Type     Protocol  Port   Source          Purpose         │
├──────────────────────────────────────────────────────────┤
│ HTTPS    TCP        443   0.0.0.0/0       API traffic     │
│ HTTP     TCP         80   0.0.0.0/0       Redirect to HTTPS │
│ SSH      TCP         22   YOUR_IP/32      Admin access only │
└──────────────────────────────────────────────────────────┘

OUTBOUND RULES — EC2 App Server (keep defaults)

┌──────────────────────────────────────────────────────────┐
│ All traffic  All  All  0.0.0.0/0  Allow all outbound      │
└──────────────────────────────────────────────────────────┘

INBOUND RULES — RDS Security Group

┌──────────────────────────────────────────────────────────┐
│ PostgreSQL  TCP  5432  sg-xxxx (EC2's security group)  Only │
│                        allow EC2 to connect to DB         │
└──────────────────────────────────────────────────────────┘
```

What should NOT be open:

- Port 3000 (NestJS) — only accessible via Nginx on 443
- Port 5432 (PostgreSQL) — only your EC2's security group
- Port 6379 (Redis) — Redis is on the same EC2, no external access needed
- Port 6432 (PgBouncer) — internal only

Part 6 — RDS Configuration (Optimize What You Have)

Set These PostgreSQL Parameters (via RDS Parameter Group)

```
AWS Console → RDS → Parameter Groups → Create Parameter Group
→ Family: postgres16 (or your version)
→ Name: rupeeflow-prod-params

Set these parameters:
```

Parameter	Value	Why
`max_connections`	100	PgBouncer handles app connections, RDS only needs 100
`shared_buffers`	2GB	25% of your RDS RAM (for db.t3.large = 8GB RAM)
`effective_cache_size`	6GB	75% of RAM — tells planner how much OS cache is available

`work_mem`	64MB	Memory per sort/hash operation
`maintenance_work_mem`	512MB	For VACUUM, index creation
`checkpoint_completion_target`	0.9	Spread checkpoint writes over 90% of interval
`wal_buffers`	64MB	Write-ahead log buffers
`default_statistics_target`	200	Better query plans for complex analytics queries
`log_min_duration_statement`	500	Log queries > 500ms (matches TypeORM maxQueryExecutionTime)
`log_connections`	off	Reduces log noise (PgBouncer generates many connections)
`autovacuum_max_workers`	4	More aggressive VACUUM for high-write tables
`autovacuum_naptime`	30	Vacuum every 30 seconds (not 60)
`random_page_cost`	1.1	For SSD storage — tells planner SSDs are fast
`effective_io_concurrency`	200	Parallel disk reads (SSD-optimized)

Enable Performance Insights (Free Tier Available)

```
AWS Console → RDS → Your instance → Modify
→ Performance Insights: Enable
→ Retention: 7 days (free)
→ Apply immediately
```

This gives you a visual breakdown of which queries are slow and why. Essential for debugging the issues from the architecture audit.

Automated Backups (Replace Multi-AZ failover)

```
AWS Console → RDS → Your instance → Modify
→ Backup retention period: 7 days
→ Backup window: 02:00-03:00 UTC (3:30-4:30 AM IST — low traffic)
→ Enable deletion protection: YES ← very important
```

Part 7 — S3 Optimization (KYC Documents)

You're using S3 for KYC document uploads. Check your bucket:

Enable Intelligent-Tiering (Saves 40–68% on storage)

```
AWS Console → S3 → rupeeflow-kyc-bucket → Management
→ Lifecycle rules → Create rule
→ Rule name: intelligent-tiering
→ Apply to all objects
→ Transition to: Intelligent-Tiering after 0 days
```

Objects not accessed for 30 days move to cheaper storage automatically. KYC documents are uploaded once, verified once, then rarely accessed.

Fix the Security Issue (from Audit C-3)

KYC bucket must be private — no public access:

```
AWS Console → S3 → Your bucket → Permissions
→ Block all public access: ON (all 4 checkboxes)
→ Bucket policy: remove any s3:GetObject for "*" principal
```

Access KYC documents only via presigned URLs (you're already doing this ✓).

Part 8 — SES / SNS Cost Optimization

SES (Email)

- First 62,000 emails/month from EC2: FREE
- Above 62K: $0.10 per 1,000 emails
- You're likely in the free tier unless sending very high volumes

Set up SES sending limits alarm:

```
aws cloudwatch put-metric-alarm \
  --alarm-name "SES-Daily-Sending-Limit" \
  --metric-name "Send" \
  --namespace "AWS/SES" \
  --statistic Sum \
  --period 86400 \
  --threshold 50000 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:ap-south-1:YOUR_ACCOUNT:billing-alerts
```

SNS (SMS OTPs)

SMS is the most expensive part of SNS. India transactional SMS:

- ~$0.01451 per SMS via AWS SNS
- If you send 10,000 OTPs/day = $145/day = **$4,350/month**

This could be a major cost driver. Check immediately:

```
AWS Console → SNS → Text messaging (SMS) → View SMS spending
```

If SMS cost is high:

1. Add OTP rate limiting (see S-2 from audit — throttle send-otp endpoint)
2. Consider switching to a cheaper SMS provider (Msg91, Kaleyra) for India — ~40–60% cheaper than AWS SNS for India routes
3. Cache OTP validation results to avoid resends

Part 9 — AWS Billing Alerts (Set These Up Now)

You should never be surprised by an AWS bill. Set three alarms:

Alert 1: Monthly budget threshold

```
AWS Console → Billing → Budgets → Create budget
→ Budget type: Cost budget
→ Period: Monthly
→ Budgeted amount: $500
→ Alert threshold 1: 80% actual ($400) → email you
→ Alert threshold 2: 100% actual ($500) → email + SMS
→ Alert threshold 3: 120% forecasted ($600) → email + SMS
```

Alert 2: RDS storage running out

```
AWS Console → CloudWatch → Alarms → Create alarm
→ Metric: RDS → Per-database metrics → FreeStorageSpace
→ Threshold: less than 10 GB
→ Period: 5 minutes
→ Action: SNS notification to your email
```

Alert 3: EC2 CPU sustained high

```
AWS Console → CloudWatch → Alarms → Create alarm
→ Metric: EC2 → Per-instance metrics → CPUUtilization
→ Threshold: greater than 85%
→ For: 3 consecutive 5-minute periods
→ Action: SNS notification
```

Part 10 — Cost After All Optimizations

```
BEFORE (current)                    AFTER OPTIMIZATION
──────────────────────────────────────────────────────
RDS Multi-AZ       ~$400/month  →  RDS Single-AZ  ~$100/month
NAT Gateway        ~$100/month  →  Deleted         $0/month
EC2 m6a.large      ~$73/month   →  EC2 Reserved    ~$44/month
RDS Storage gp2    ~$50/month   →  gp3             ~$35/month
CloudWatch Logs    ~$40/month   →  30-day retention ~$8/month
S3 KYC             ~$20/month   →  Intelligent-tier ~$12/month
SES Email          ~$10/month   →  Same             ~$10/month
SNS SMS            varies       →  Rate limited + cheaper provider
Data Transfer      ~$80/month   →  Optimized        ~$40/month
──────────────────────────────────────────────────────
TOTAL              ~$773/month  →  TOTAL           ~$249/month

SAVINGS            ~$524/month = $6,288/year
```

Part 11 — Step-by-Step This Week

Do these in this exact order. Each step is reversible.

```
Day 1 (30 mins):
  ☐ Go to Cost Explorer → find exact breakdown of $900
  ☐ Set CloudWatch log retention to 30 days on all log groups
  ☐ Set up billing alerts at $400 and $500

Day 2 (1 hour, 2 AM IST maintenance window):
  ☐ Modify RDS: gp2 → gp3 storage type (no downtime)
  ☐ Modify RDS: enable Performance Insights (no downtime)
  ☐ Set RDS backup window to 02:00-03:00 UTC

Day 3 (schedule maintenance window, 15 min downtime):
  ☐ Modify RDS: Multi-AZ → Single-AZ (1-2 min downtime)
  ☐ Tell your team: "DB unavailable 2:00-2:15 AM IST Sunday"

Day 4 (1 hour):
  ☐ Check if NAT Gateway exists in VPC
  ☐ SSH from EC2 → test psql connection to RDS via hostname
  ☐ If connection works without NAT: delete NAT Gateway
  ☐ Release the associated Elastic IP

Day 5 (2 hours):
  ☐ Create and attach RDS Parameter Group with the settings from Part 6
  ☐ Apply parameter group (requires DB restart — 2-3 min downtime)
  ☐ Check SNS SMS spend — if > $50/month, add rate limiting

Day 6:
  ☐ Verify Nginx config (Part 4) is correct
  ☐ Verify EC2 security group (port 3000 not publicly open)
  ☐ Verify S3 bucket: all public access blocked
  ☐ Purchase EC2 Reserved Instance (1-year) after confirming savings

Day 7:
  ☐ Check new estimated bill in Cost Explorer
  ☐ Confirm all alerts are firing correctly on test thresholds
```

Part 12 — When to Add More AWS Resources

Don't add resources until you hit these thresholds:

```
METRIC                    ADD THIS WHEN...              ADD THIS
──────────────────────────────────────────────────────────────────
EC2 CPU avg > 70%          Sustained for 1 week         Second EC2 + ALB
RDS CPU avg > 60%          Sustained for 1 week         Upgrade instance type
RDS FreeStorageSpace < 20GB  Any time                   Add 100GB storage
Redis memory > 70%         Sustained for 3 days         Upgrade EC2 RAM
```

```
p99 API latency > 1s      Any time in production       Profile first (don't guess)
Payin queue depth > 5000  Sustained for 30 mins        Add a second EC2 for workers
```

Before spending money: always profile first. Use RDS Performance Insights and PM2 logs to find the actual bottleneck. In 80% of cases, a missing index or a slow query is the problem — not insufficient infrastructure.

Quick Reference — Monthly Costs After Optimization

Service	Instance/Config	Monthly Cost
EC2 m6a.large	1-year reserved	~$44
RDS PostgreSQL	db.t3.large Single-AZ, gp3 100GB	~$75
Redis	Docker on EC2 (same instance)	$0
S3	Intelligent-Tiering, ~50GB KYC docs	~$8
SES	< 62K emails/month (from EC2)	$0
SNS SMS	After rate limiting	varies
CloudWatch	30-day retention	~$8
Data Transfer	Optimized	~$40
Route 53	Hosted zone	~$0.50
TOTAL		**~$175 + SNS**

End of Document — Rupeeflow AWS Optimization Guide — March 2026 Expected savings: $500–600/month after implementing all fixes